Thadomal Shahani Engineering College, Bandra
Department of Artificial Intelligence & Data Science

Name:Rashi Bhatia				**Subject: ASD&DA**
TE - 1	**Batch:T11**	**Sem: 5**	**Roll No:12**	**Date:14/07/2026**

Experiment 1

AIM : To perform version control operations using Git.

LO Mapped : 2

LO : Use Git efficiently for version control source code development and collaborate developemt

THEORY : Version Control is a system that helps developers track, manage, and record changes made to files and source code over time. It allows users to maintain different versions of a project, making it easy to review changes, restore previous versions, and collaborate with multiple developers without overwriting each other's work.

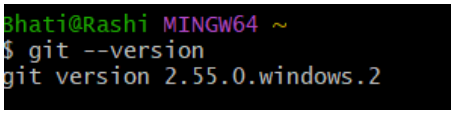
Version control systems keep a complete history of modifications, including who made the changes, when they were made, and why they were made. This improves project management, reduces the risk of data loss, and enables efficient teamwork.

Git is one of the most popular distributed version control systems. It stores the complete project history locally, allowing developers to work offline and synchronize changes with remote repositories such as GitHub when needed.

OUTPUT :

1.git --version

Displays the installed Git version.



Name:Rashi Bhatia				Subject: ASD&DA
TE - 1	Batch:T11	Sem: 5	Roll No:12	Date:14/07/2026

2. git config --global user.name

This command sets the global username for Git. Every commit made from the computer will contain this username.

Syntax

git config --global user.name "Rashi Bhatia"

```
Bhati@Rashi MINGW64 ~
$ git config --global user.name 'Rashi Bhatia'
```

3. git config --global user.email

This command sets the global email address associated with Git commits.

Syntax

git config --global user.email bhatiarashi19@gmail.com

```
Bhati@Rashi MINGW64 ~
$ git config --global user.name "bhatiarashi19@gmail.com"
```

4. git config --list

Displays all Git configuration settings such as username, email, default branch, and credential helper.

Syntax

git config --list

Name:Rashi Bhatia				Subject: ASD&DA
TE - 1	Batch:T11	Sem: 5	Roll No:12	Date:14/07/2026

```
Bhati@Rashi MINGW64 ~
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=main
user.name=Rashi Bhatia
user.email=bhatiarashi19@gmail.com
```

5. mkdir GitDemo

Creates a new project directory named GitDemo.

Syntax

mkdir GitDemo

```
Bhati@Rashi MINGW64 ~
$ mkdir GitDemo
```

6. cd GitDemo

Changes the current working directory to the GitDemo folder.

Syntax

cd GitDemo

```
Bhati@Rashi MINGW64 ~
$ cd GitDemo
```

7. git init

Initializes a new Git repository by creating a hidden .git directory.

Name:Rashi Bhatia				Subject: ASD&DA
TE - 1	Batch:T11	Sem: 5	Roll No:12	Date:14/07/2026

Syntax

git init

```
Bhati@Rashi MINGW64 ~/GitDemo
$ git init
Initialized empty Git repository in C:/Users/Bhati/GitDemo/.git/
```

8. git status

Displays the current status of the repository including tracked, modified, and untracked files.

Syntax

git status

```
Bhati@Rashi MINGW64 ~/GitDemo (main)
$ git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

9. Create hello.py

Creates a Python source file that will be tracked by Git.

Code : print("Hello Git")

Name:Rashi Bhatia				Subject: ASD&DA
TE - 1	Batch:T11	Sem: 5	Roll No:12	Date:14/07/2026

```
Bhati@Rashi MINGW64 ~/GitDemo (main)
$ touch hello.py

Bhati@Rashi MINGW64 ~/GitDemo (main)
$ notepad hello.py

Bhati@Rashi MINGW64 ~/GitDemo (main)
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        hello.py

nothing added to commit but untracked files present (use "git add" to track)
```

10. git add hello.py

Adds the Python file to the staging area before committing.

Syntax

git add hello.py

```
Bhati@Rashi MINGW64 ~/GitDemo (main)
$ git add hello.py

Bhati@Rashi MINGW64 ~/GitDemo (main)
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   hello.py
```

11. git commit -m "Initial commit"

Creates the first snapshot (version) of the project in the repository.

Syntax

Name:Rashi Bhatia				Subject: ASD&DA
TE - 1	Batch:T11	Sem: 5	Roll No:12	Date:14/07/2026

git commit -m "Initial commit"

```
Bhati@Rashi MINGW64 ~/GitDemo (main)
$ git commit -m "Initial commit"
[main (root-commit) ad32f18] Initial commit
 1 file changed, 1 insertion(+)
 create mode 100644 hello.py
```

12. git log

Displays the complete commit history including commit ID, author, date, and message.

Syntax

git log

```
Bhati@Rashi MINGW64 ~/GitDemo (main)
$ git log
commit ad32f189b3beed1cc77aff1020f8b9020a522fe8 (HEAD -> main)
Author: Rashi Bhatia <bhatiarashi19@gmail.com>
Date:   Tue Jul 14 20:21:52 2026 +0530

    Initial commit
```

13. git log --oneline

Displays the commit history in a concise single-line format.

Syntax

git log --oneline

```
Bhati@Rashi MINGW64 ~/GitDemo (main)
$ git log --oneline
ad32f18 (HEAD -> main) Initial commit
```

Name:Rashi Bhatia				Subject: ASD&DA
TE - 1	Batch:T11	Sem: 5	Roll No:12	Date:14/07/2026

14. git diff

Shows the differences between the working directory and the last committed version.

Syntax

git diff

```
Bhati@Rashi MINGW64 ~/GitDemo (main)
$ notepad hello.py

Bhati@Rashi MINGW64 ~/GitDemo (main)
$ git diff
diff --git a/hello.py b/hello.py
index b4c183a..a0d5b62 100644
--- a/hello.py
+++ b/hello.py
@@ -1 +1,3 @@
-print("Hello Git")
\ No newline at end of file
+print("Hello Git")
+
+print("This is my second version")
\ No newline at end of file

Bhati@Rashi MINGW64 ~/GitDemo (main)
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   hello.py

no changes added to commit (use "git add" and/or "git commit -a")
```

15. git add hello.py

Stages the modified Python file for the next commit.

Syntax

git add hello.py

```
Bhati@Rashi MINGW64 ~/GitDemo (main)
$ git add hello.py
```

Name:Rashi Bhatia				Subject: ASD&DA
TE - 1	Batch:T11	Sem: 5	Roll No:12	Date:14/07/2026

16. git commit -m "Update hello.py"

Creates a new version of the project after modifying the Python file.

Syntax

git commit -m "Update hello.py"

```
Bhati@Rashi MINGW64 ~/GitDemo (main)
$ git commit -m "Update hello.py"
[main 9eeea70] Update hello.py
 1 file changed, 3 insertions(+), 1 deletion(-)
```

17. git remote add origin

Links the local Git repository with a remote repository hosted on GitHub.

Syntax

git remote add origin https://github.com/bhatiarashi19-ops/GitDemo.git

```
Bhati@Rashi MINGW64 ~/GitDemo (main)
$ git remote add origin https://github.com/bhatiarashi19-ops/GitDemo.get
```

18. git remote -v

Displays the remote repository URLs used for fetching and pushing changes.

Syntax

git remote -v

```
Bhati@Rashi MINGW64 ~/GitDemo (main)
$ git remote -v
origin  https://github.com/bhatiarashi19-ops/GitDemo.git (fetch)
origin  https://github.com/bhatiarashi19-ops/GitDemo.git (push)
```

Name:Rashi Bhatia				Subject: ASD&DA
TE - 1	Batch:T11	Sem: 5	Roll No:12	Date:14/07/2026

19. git push -u origin main

Uploads the local repository and commits to the GitHub repository.

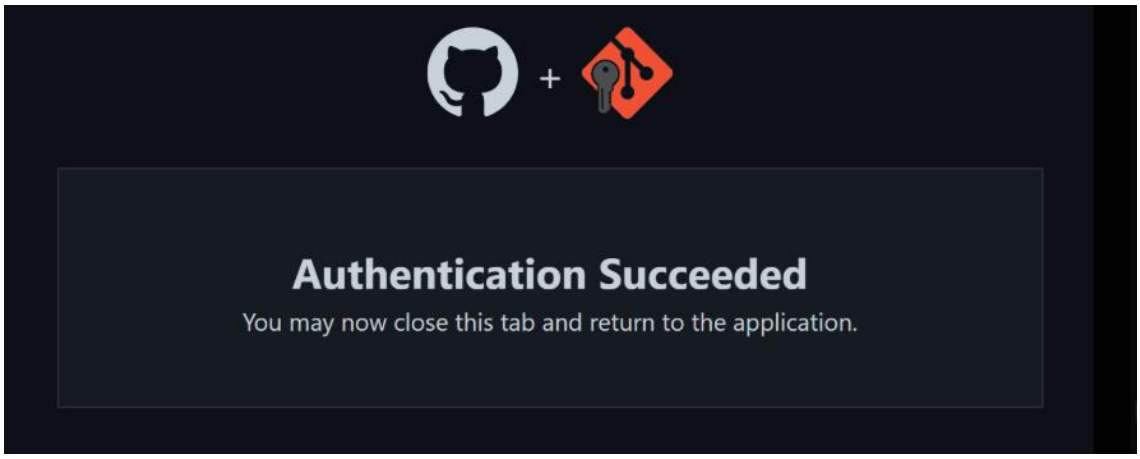
Syntax

git push -u origin main

```
Bhati@Rashi MINGW64 ~/GitDemo (main)
$ git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (6/6), 501 bytes | 501.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/bhatiarashi19-ops/GitDemo.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

20. GitHub Authentication

Authenticates the user with GitHub using the browser so Git can securely access the remote repository.



Name:Rashi Bhatia				Subject: ASD&DA
TE - 1	Batch:T11	Sem: 5	Roll No:12	Date:14/07/2026

21. git pull origin main

Downloads and merges the latest changes from the remote repository.

Syntax

git pull origin main

```
Bhati@Rashi MINGW64 ~/GitDemo (main)
$ git pull origin main
From https://github.com/bhatiarashi19-ops/GitDemo
 * branch            main       -> FETCH_HEAD
Already up to date.
```

22. git branch feature

Creates a new branch named feature for independent development.

Syntax

git branch feature

```
Bhati@Rashi MINGW64 ~/GitDemo (main)
$ git branch feature

Bhati@Rashi MINGW64 ~/GitDemo (main)
$ git branch
  feature
* main
```

23. git checkout feature

Switches from the main branch to the feature branch.

Syntax

git checkout feature

Name:Rashi Bhatia				Subject: ASD&DA
TE - 1	Batch:T11	Sem: 5	Roll No:12	Date:14/07/2026

```
Bhati@Rashi MINGW64 ~/GitDemo (feature)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

24. git merge feature

Merges the changes from the feature branch into the current branch.

Syntax

git merge feature

```
Bhati@Rashi MINGW64 ~/GitDemo (main)
$ git merge feature
Already up to date.
```

25. git clone

Creates a local copy of the GitHub repository on the computer.

Syntax

git clone https://github.com/bhatiarashi19-ops/GitDemo.git GitDemoClone

```
Bhati@Rashi MINGW64 ~
$ git clone https://github.com/bhatiarashi19-ops/GitDemo.git GitDemoClone
Cloning into 'GitDemoClone'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 6 (delta 0), reused 6 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (6/6), done.
```

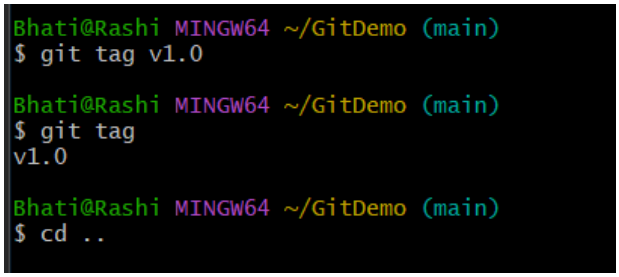
Name:Rashi Bhatia				Subject: ASD&DA
TE - 1	Batch:T11	Sem: 5	Roll No:12	Date:14/07/2026

26. git tag v1.0

Creates a tag named v1.0 to mark a specific version of the project.

Syntax

git tag v1.0



CONCLUSION : Successfully performed Version Control Operations using Git and GitHub including repository creation, configuration, staging, committing, branching, merging, pushing, pulling, cloning, tagging, and remote repository management.